

APRESENTAÇÃO DE PROJETO

Web Amigos

Plataforma de E-Learning & Venda de Cursos Online



PROJETO:	Plataforma Web para Venda de Cursos (Full-Stack)
EQUIPE:	Eduardo Vinicius Faleiro, Francisco de Paula Lemos, Lucas dos Santos Santin, Lucas Gusmão Valduga, Marlon Correia
TECNOLOGIAS:	Go (Chi, GORM), React (TypeScript, Vite, MUI), PostgreSQL
DATA:	Junho de 2026

APRESENTAÇÃO GERAL

Objetivo e Escopo do Sistema

O projeto **Web Amigos** é uma plataforma integrada de e-learning e comércio digital de cursos. Ela fornece uma solução completa para que criadores de conteúdo publiquem e monetizem suas aulas, enquanto estudantes usufruem de um ambiente de aprendizado seguro, interativo e fluido.

C Perspectiva do Criador

Acesso a uma área administrativa para gerenciar o ciclo de vida dos cursos. Permite cadastrar cursos, estruturar a ementa em módulos e lições, gerenciar preços e monitorar o status de publicação.

A Perspectiva do Aluno

Navegação pelo catálogo público de cursos publicados, realização de compras seguras, consumo de aulas em vídeo, publicação de avaliações gerais e participação ativa no fórum de discussão de cada aula.

T Pilares de Desenvolvimento

A arquitetura do sistema foi projetada focando em **segurança de transações** (matrícula sincronizada via webhooks de pagamento), **controle rígido de acessos** (permissões baseadas em papéis) e **alta performance relacional** para gerenciar interações e comentários.

INTEGRAÇÃO TÉCNICA

Fluxo de Dados por Caso de Uso

Abaixo descrevemos o fluxo de integração do Frontend para a API REST e a manipulação do banco de dados PostgreSQL em cada um dos escopos da aplicação:

1 Cadastro e Login

O usuário envia os dados. A API valida o e-mail, criptografa a senha com bcrypt e insere na tabela users. No login, a API valida a credencial, gera um token JWT e o Frontend armazena no localStorage.

2 Gerência de Usuários

Administradores gerenciam usuários. O Frontend envia a requisição HTTP com o token JWT. A API intercepta, valida se o solicitante possui papel admin e atualiza a coluna role no PostgreSQL.

3 Gestão de Cursos

Criadores e administradores estruturam ementas. O Frontend envia metadados. A API valida a role (creator/admin) e persiste as entidades nas tabelas courses, modules e lessons.

4 Compra e Liberação

O aluno inicia a compra. A tabela enrollments cria o registro como pending. Ao confirmar pagamento, o gateway chama o webhook da API, que altera o status para active e libera o acesso.

5 Avaliação de Cursos

Alunos ativos enviam notas (1 a 5) e comentários. A API garante que o usuário possui matrícula ativa no curso e que ainda não o avaliou, inserindo a review na tabela course_reviews.

6 Fórum de Dúvidas

Alunos comentam nas aulas. O Frontend envia a dúvida vinculada à aula. A API valida a matrícula ativa e insere a mensagem na tabela indexada lesson_comments para exibição em tempo real.

REGRAS DE NEGÓCIO E CAMADAS

Políticas de Acesso e Implementação por Camadas

1. Controle de Acesso Baseado em Papéis (RBAC)

O sistema implementa papéis (roles) rígidos para garantir a segurança operacional e de propriedade intelectual. A verificação ocorre de forma espelhada no cliente e no servidor:

Papel (Role)	Permissões no Frontend	Validação no Backend (API)
Admin	Acesso total. Painel de controle de usuários e alteração de papéis. Gerenciamento geral de cursos.	Apenas requisições com <code>userRole == "admin"</code> podem acessar <code>PUT /users/{id}/role</code> .
Creator	Menu do Criador, CRUD de cursos próprios, adição de módulos e aulas. Sem acesso à gestão de usuários.	Handlers de criação/alteração validam se o ID do criador coincide ou se o papel é criador/admin.
Student	Catálogo público, área "Meus Cursos", player de vídeo, postagem de comentários e avaliações.	Validado pelo <code>HasActiveEnrollment</code> do banco ao tentar ler as aulas e postar comentários.

2. Validação Contígua de Regras de Negócio

- **Unicidade de Avaliações:** Um estudante só pode criar 1 review por curso. O `ReviewService` valida isso consultando se já existe um registro correspondente na tabela `course_reviews` para o par `(user_id, course_id)` antes de prosseguir.
- **Moderação de Fóruns:** Um comentário em aula (`lesson_comments`) só pode ser excluído por seu próprio autor, por qualquer criador do curso correspondente, ou por um administrador. O handler de deleção faz essa validação de escopo buscando a aula associada.
- **Acesso Condicional:** Creators e admins têm acesso automático a todo o conteúdo dos cursos cadastrados, pulando a validação de matrícula ativa do `EnrollmentService`.

```
// Exemplo de Validação de Permissão de Edição no Backend Go userRole :=
RetrieveUserRole(r.Context()) if userRole != "admin" && userRole != "creator" {
w.WriteHeader(http.StatusUnauthorized) return }
```

ENGENHARIA DE DADOS

Modelagem de Banco de Dados & Conclusão

Para suportar todas as regras operacionais, o banco de dados PostgreSQL estruturado conta com 7 tabelas normalizadas. O GORM do backend cuida da migração automatizada (auto-migration) ao subir o contêiner Docker.

Tabelas e Relacionamentos no PostgreSQL

Tabela	Chave Primária (PK)	Chaves Estrangeiras (FK)	Constraints e Detalhes Importantes
users	id (UUID)	Nenhuma	email é único e indexado. password_hash armazena senhas em bcrypt.
courses	id (UUID)	creator_id → users.id	Preço em decimal(10,2). ID do produto integrado ao gateway de pagamento.
modules	id (UUID)	course_id → courses.id	Campo position para controlar a ordem de exibição na ementa.
lessons	id (UUID)	module_id, course_id	Contém links de vídeo (YouTube ID) e flag is_free para aulas preview.
enrollments	id (UUID)	user_id, course_id	Status da matrícula e data de expiração (expires_at).
course_reviews	id (UUID)	course_id, user_id	Valores de rating entre 1 e 5. Unicidade pelo par usuário-curso.
lesson_comments	id (UUID)	lesson_id, user_id	Fórum de interação direta. Indexado para carregamento ágil por lição.

Conclusão

O projeto **Web Amigos** fornece uma solução robusta e moderna de e-learning. A sinergia entre o frontend SPA em React e o backend Go estruturado garante segurança no processamento, velocidade nas requisições, baixa latência de banco de dados e uma experiência de uso consistente para criadores, alunos e administradores.